30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 13)

“Avoiding Priority Inversion”

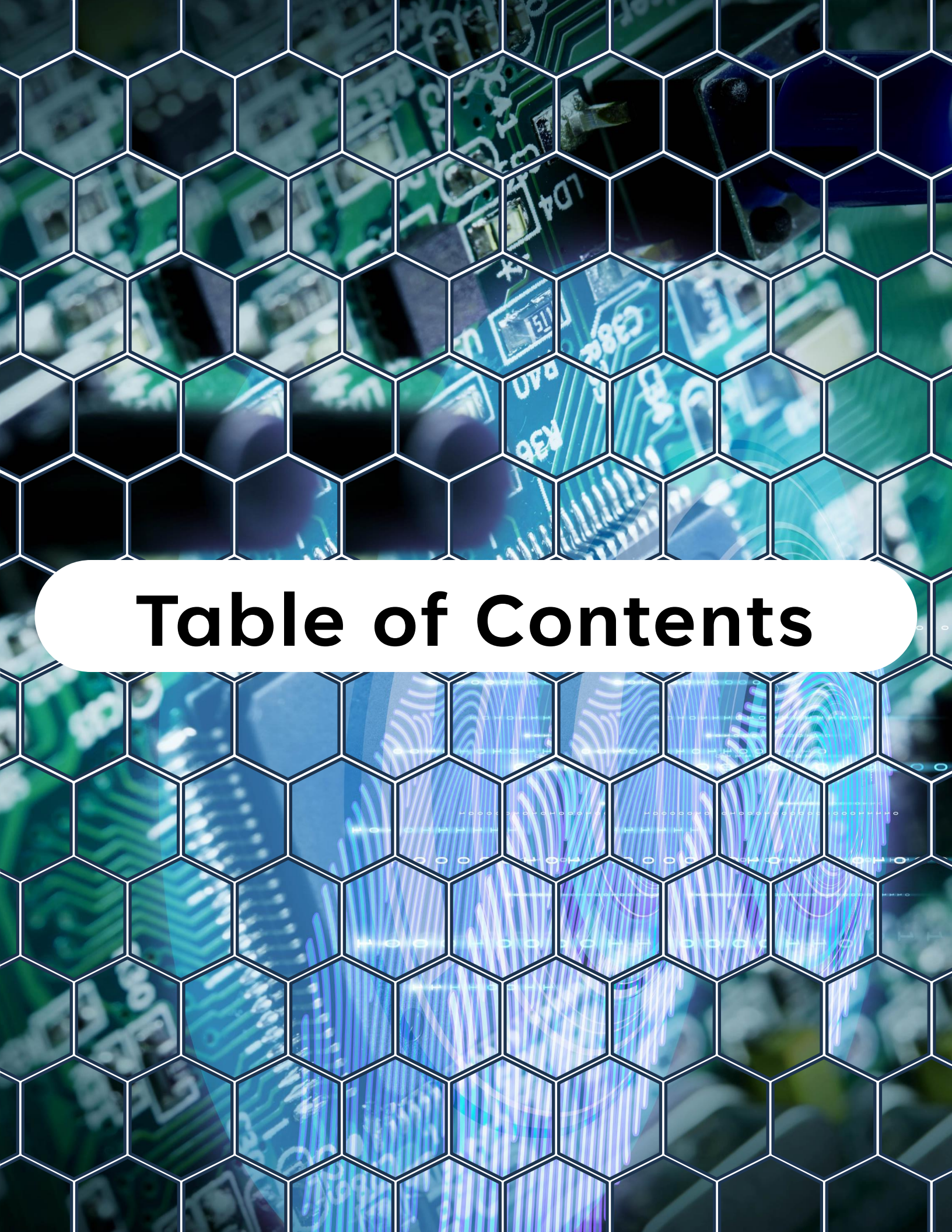
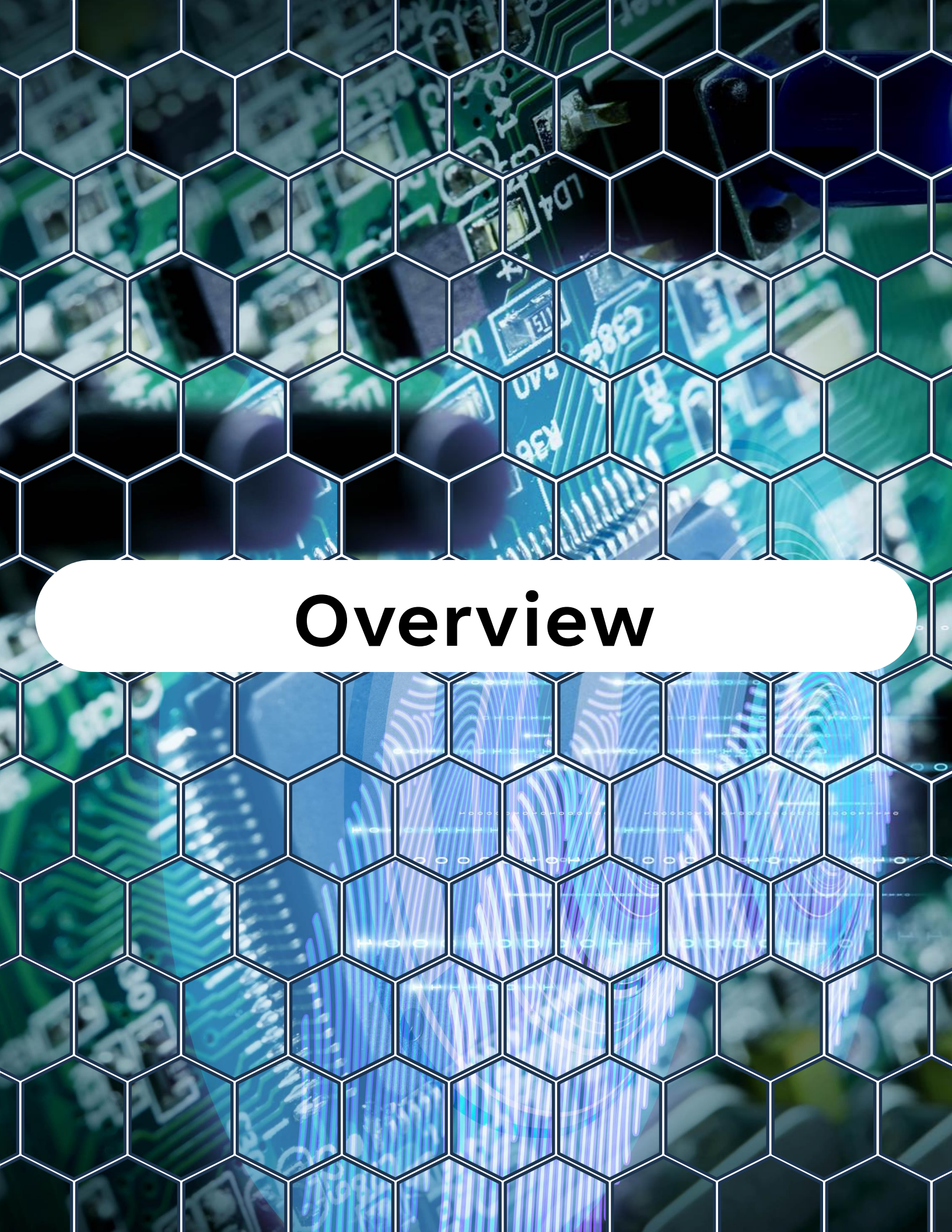


Table of Contents

Table of Contents



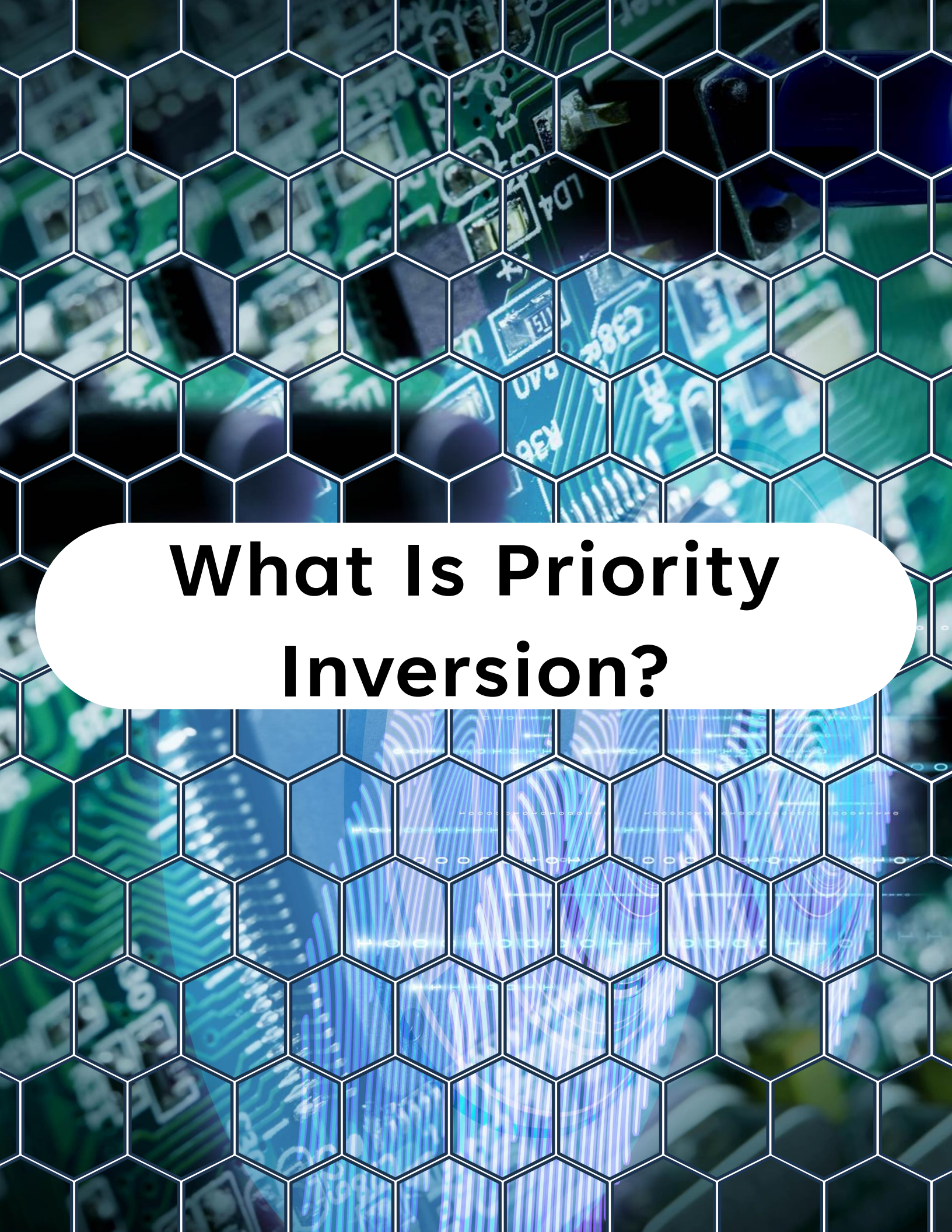
Overview

Overview

On **Day 13**, you'll learn:

- What priority inversion is and why it's dangerous in real-time systems
- A classic priority inversion scenario
- How FreeRTOS handles priority inversion with priority inheritance
- Design patterns and best practices to avoid it
- A practical ESP32 example demonstrating the issue and solution

By the end of this lesson, you'll know how to identify and prevent priority inversion in your FreeRTOS projects.



What Is Priority Inversion?

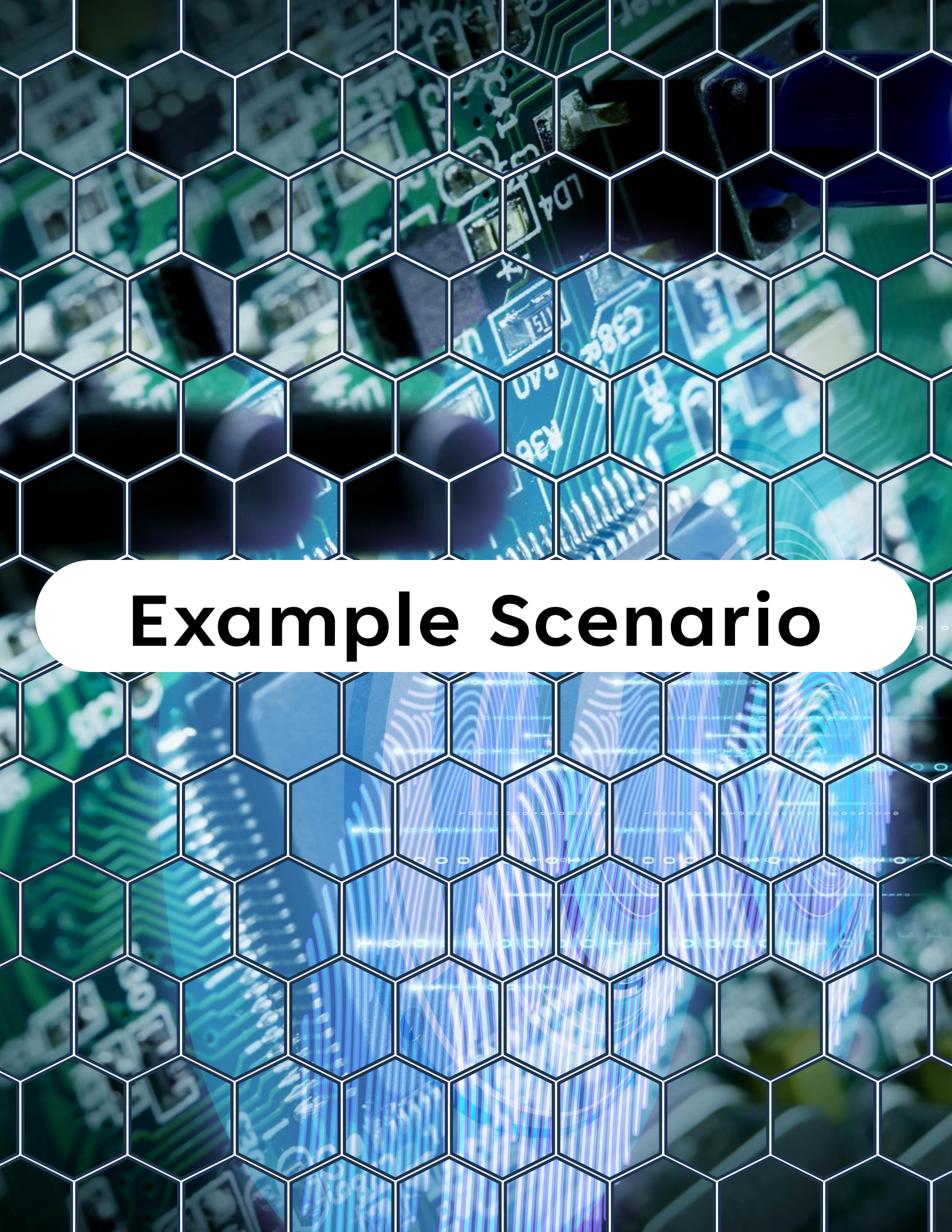
What Is Priority Inversion?

Priority inversion occurs when:

- A **low-priority task** holds a resource (e.g., a mutex).
- A **high-priority task** needs the same resource but is blocked, waiting.
- Meanwhile, a **medium-priority task** keeps running because it doesn't depend on the resource.

📌 **Result:**

the high-priority task is "**inverted**" below the medium-priority task, breaking real-time guarantees.



Example Scenario

Example Scenario

Imagine:

- **Task L (Low priority):** Prints to UART while holding a mutex.
- **Task H (High priority):** Needs UART mutex to log critical data.
- **Task M (Medium priority):** Performs CPU work but doesn’t use the mutex.

Without protection:

- Task H gets blocked by Task L.
- Task M preempts Task L.
- Task H waits unnecessarily until Task M finishes → **priority inversion**.

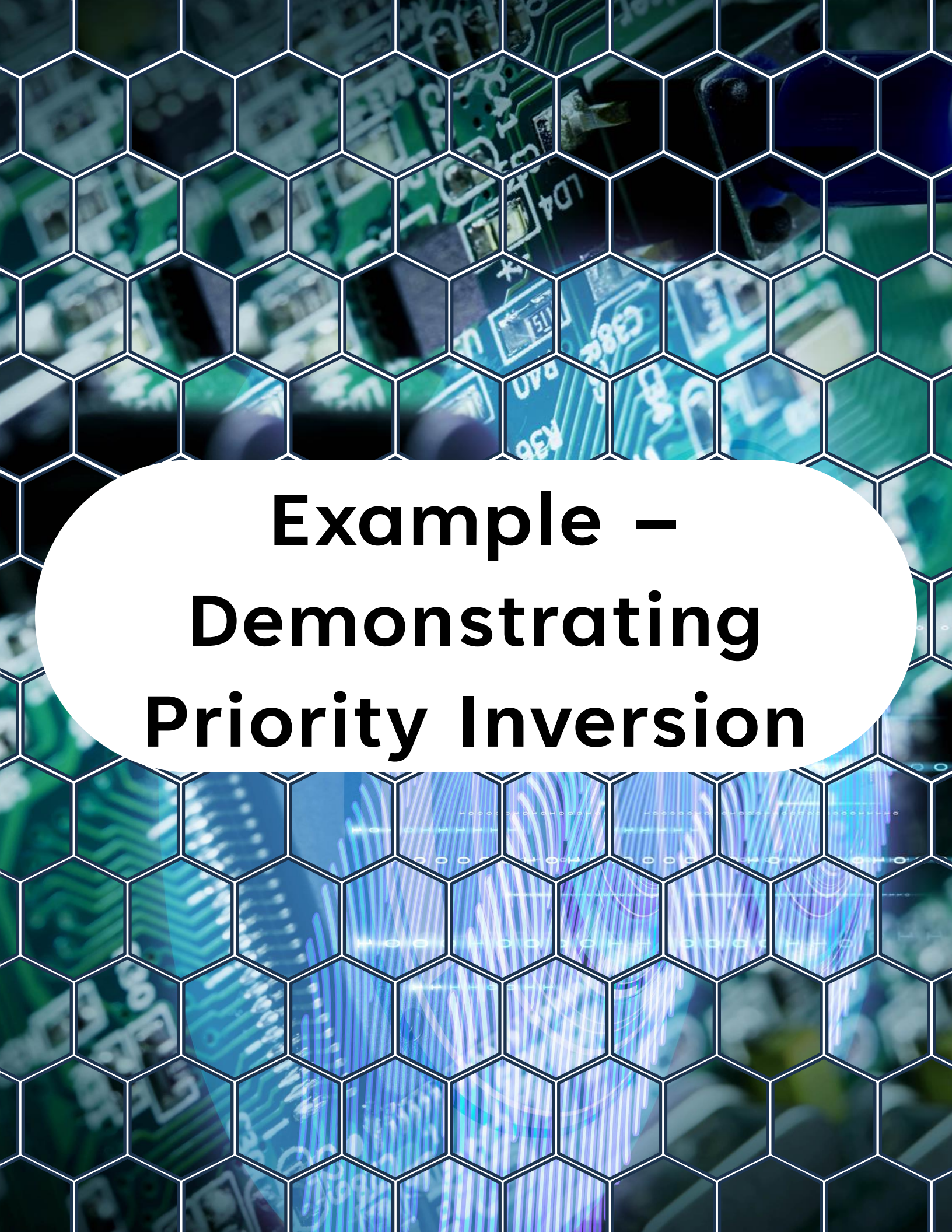
FreeRTOS Solution: Priority Inheritance

FreeRTOS Solution: Priority Inheritance

FreeRTOS mutexes implement **priority inheritance**:

- When a high-priority task is waiting on a mutex held by a lower-priority task, the lower-priority task's priority is **temporarily boosted**.
- This ensures the resource is released quickly.
- Once released, the task returns to its original priority.

✅ This prevents medium-priority tasks from interfering.



Example – Demonstrating Priority Inversion

Example – Demonstrating Priority Inversion

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"

// Mutex for resource sharing
SemaphoreHandle_t uart_mutex;

// Low priority task that holds a resource
void low_task(void *pv) {
    while (1) {
        if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
            printf("Low task acquired UART\n");
            vTaskDelay(pdMS_TO_TICKS(2000)); // Hold resource
            printf("Low task releasing UART\n");
            xSemaphoreGive(uart_mutex);
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// High priority task that needs the same resource
void high_task(void *pv) {
    while (1) {
        vTaskDelay(pdMS_TO_TICKS(500)); // Start later
        if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
            printf("High task using UART\n");
            vTaskDelay(pdMS_TO_TICKS(500));
            xSemaphoreGive(uart_mutex);
        }
    }
```

Example – Demonstrating Priority Inversion

```
}

// High priority task that needs the same resource
void high_task(void *pv) {
    while (1) {
        vTaskDelay(pdMS_TO_TICKS(500)); // Start later
        if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
            printf("High task using UART\n");
            vTaskDelay(pdMS_TO_TICKS(500));
            xSemaphoreGive(uart_mutex);
        }
    }
}

// Medium priority task to demonstrate priority inversion
void medium_task(void *pv) {
    while (1) {
        printf("Medium task running\n");
        vTaskDelay(pdMS_TO_TICKS(300));
    }
}

// Entry point
void app_main() {
    // Create the mutex
    uart_mutex = xSemaphoreCreateMutex();

    // Create tasks with different priorities
    xTaskCreate(low_task, "LowTask", 2048, NULL, 2, NULL);
    xTaskCreate(high_task, "HighTask", 2048, NULL, 10, NULL);
    xTaskCreate(medium_task, "MedTask", 2048, NULL, 5, NULL);
}
```

Example – Demonstrating Priority Inversion

Expected Behavior

Without mutex (or with a binary semaphore):

- High task waits, but medium task preempts low task → priority inversion.

With FreeRTOS mutex (with priority inheritance):

- Low task inherits high task's priority → finishes sooner.
- High task runs promptly after mutex is released.
- Medium task does not interfere.

Best Practices to Avoid Priority Inversion

Best Practices to Avoid Priority Inversion

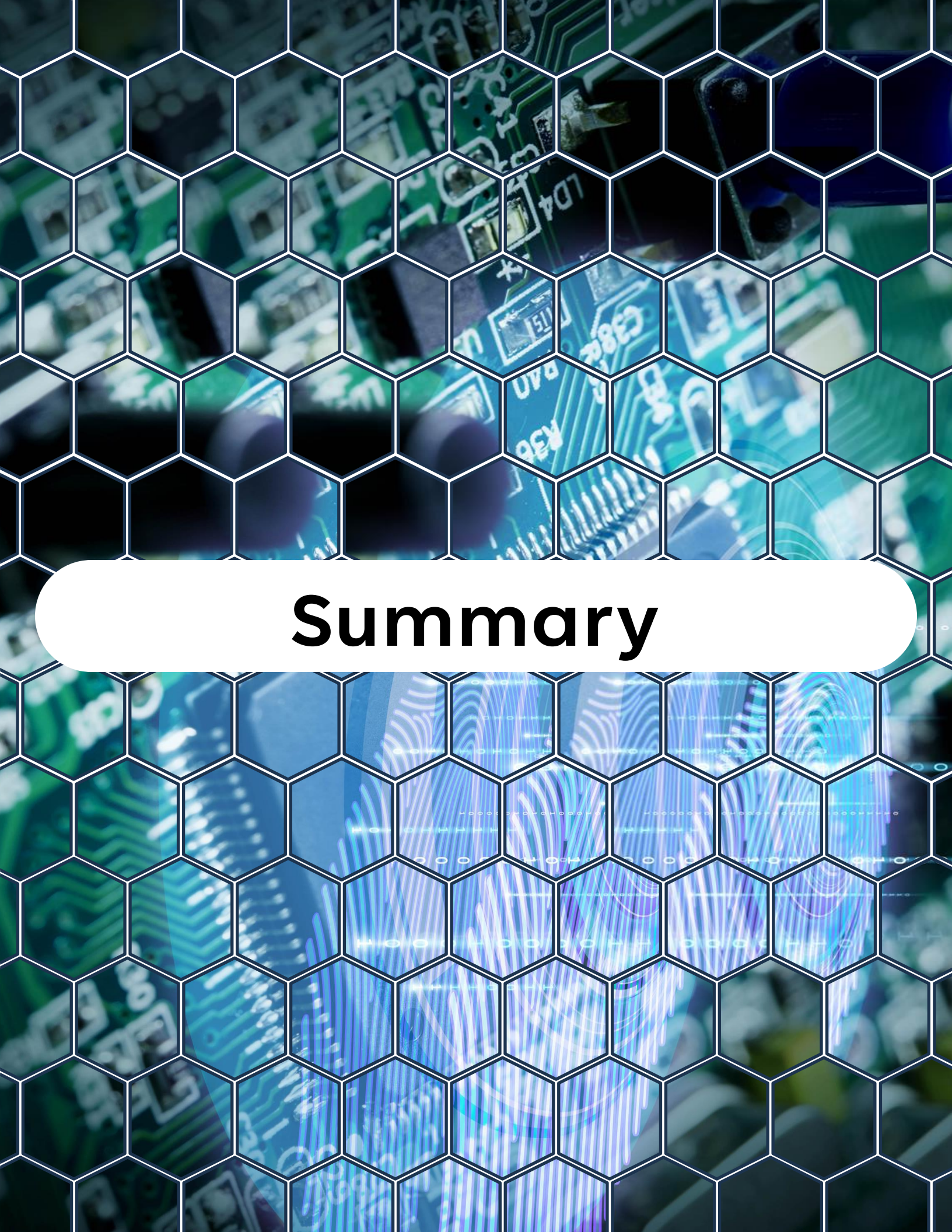
✅ Use FreeRTOS mutexes, not binary semaphores, when protecting resources shared by tasks of different priorities.

✅ Keep critical sections short — hold mutexes only as long as necessary.

✅ Avoid nested locks when possible; otherwise, consider recursive mutexes (Day 12).

✅ Give critical system tasks higher priorities than background or logging tasks.

✅ Profile and test under load to ensure latency requirements are met.

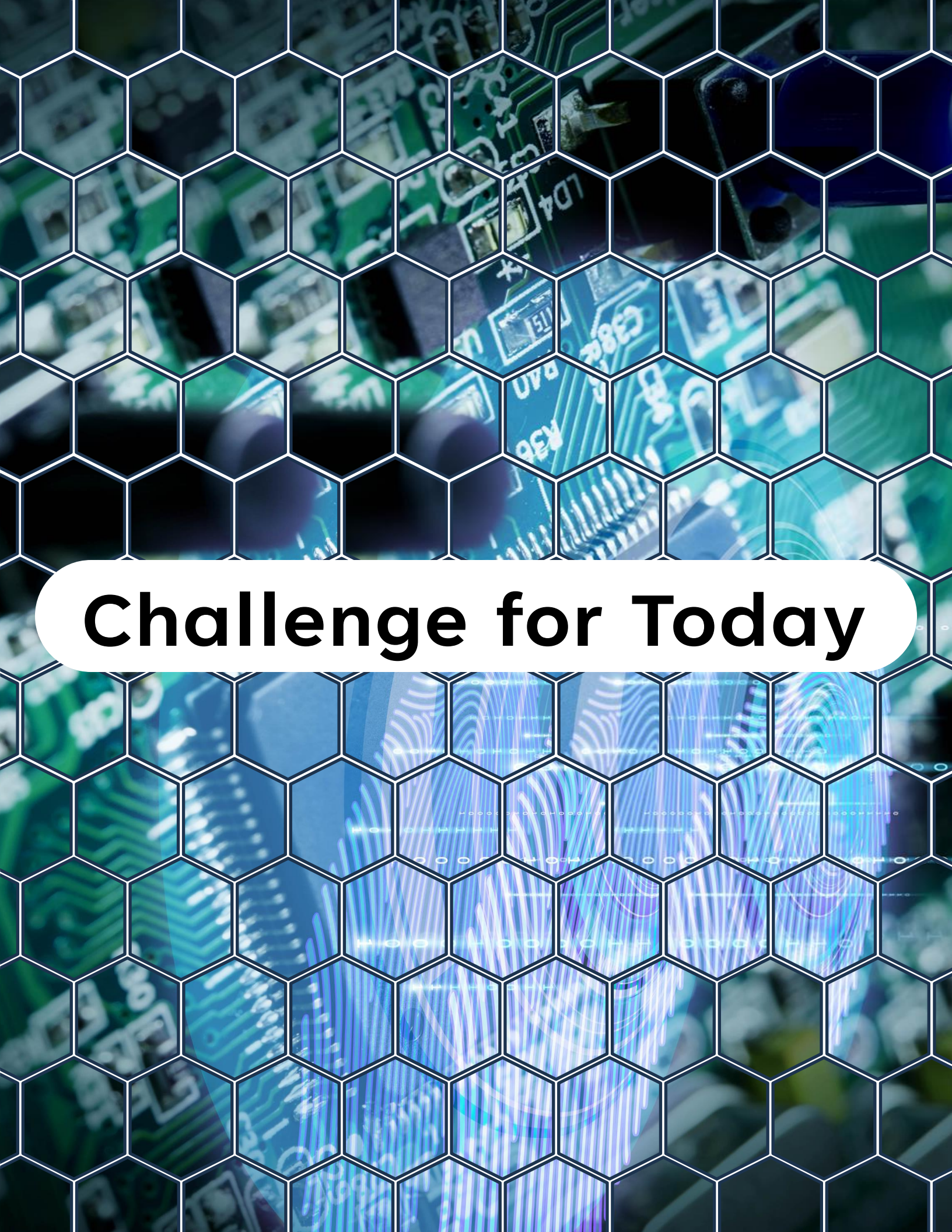


Summary

Summary

- **Priority inversion** happens when a high-priority task is blocked by a low-priority one, while a medium-priority task preempts.
- FreeRTOS **mutexes solve** this with **priority inheritance**.
- Use the right synchronization primitive:
  - **Mutexes** → shared resource protection (with inheritance).
  - **Semaphores** → event signaling.

Design task priorities and critical sections carefully to avoid bottlenecks.


Challenge for Today

Challenge for Today

Modify today's example by:

1. Replacing the mutex with a **binary semaphore**.
2. Observe how priority inversion occurs (medium task preempts low task).
3. Switch back to a mutex and confirm how priority inheritance fixes the problem.

"Thanks for watching!

If you enjoyed this video,

make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

DISCOVER, LEARN, SUCCEED!

https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil